



Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

Large language model assisted hierarchical reinforcement learning training

Qianxi Li^{a, b}, Bao Pang^{a, b, *}, Yong Song^{a, b}, Hongze Fu^a, Qingyang Xu^a,
Xianfeng Yuan^a, Xiaolong Xu^a, Chengjin Zhang^a

^a School of Mechanical, Electrical and Information Engineering, Shandong University, Weihai, 264209, China

^b Shandong Key Laboratory of Intelligent Electronic Packaging Testing and Application, Weihai, 264209, China

HIGHLIGHTS

- Proposes LLMHRL for language model-assisted hierarchical reinforcement learning.
- High-level policy receives language model guidance for improved performance.
- Decomposes high-level policies into skills and targets to reduce action space.

ARTICLE INFO

Keywords:

Reinforcement learning
Large language models
Sequential decision making

ABSTRACT

Traditional reinforcement learning (RL) cannot solve complex long-sequence decision tasks, especially when the environment rewards are sparse. Large language models (LLMs) can perform well in long-sequence decision tasks by leveraging their powerful inference capabilities. Although LLMs possess a large amount of general knowledge, LLM-based agents lack expertise in solving specific target problems. Considering that reinforcement learning models are smaller than LLMs and can be trained specifically to perform well on specific tasks, this paper proposes a hierarchical reinforcement learning framework assisted by a large language model, called LLMHRL. In this framework, the LLM acts as a teacher agent to guide the exploration of high-level policy in hierarchical reinforcement learning. The low-level policy consists of a library of selection-based policies. The agent executes specific actions based on the low-level policy chosen by the high-level policy. Furthermore, to reduce the action space of high-level policy, this paper decomposes it into skill options and target options. The two types of options are combined to obtain a high-level policy. This paper evaluates LLMHRL against baseline methods using both public and custom-built harder tasks across three environments: MiniGrid for key-door pairing, ManiSkill for tabletop sorting, and real-world scenarios. The results show that LLMHRL outperforms existing methods in success rate, convergence speed, and average return.

1. Introduction

One of the goals of modern optimal control is to develop strategies that can complete long-sequence tasks, which require an accurate understanding of the environment and strong reasoning capabilities [1]. RL has achieved great success in solving sequential

* Corresponding author at: School of Mechanical, Electrical and Information Engineering, Shandong University, Weihai, 264209, China.

Email addresses: liqianxi@mail.sdu.edu.cn (Q. Li), pangbao@sdu.edu.cn (B. Pang), songyong@sdu.edu.cn (Y. Song), fuhongze@mail.sdu.edu.cn (H. Fu), qingyangxu@sdu.edu.cn (Q. Xu), yuanxianfeng@sdu.edu.cn (X. Yuan), xuxiaolong@sdu.edu.cn (X. Xu), cjzhang@sdu.edu.cn (C. Zhang).

<https://doi.org/10.1016/j.ins.2025.122688>

Received 11 April 2025; Received in revised form 12 September 2025; Accepted 12 September 2025

Available online 17 September 2025

0020-0255/© 2025 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

decision-making problems. The optimal policy is learned through the interaction of the agent with the environment, thereby reducing the need for prior knowledge.

However, traditional RL suffers from significant performance degradation when faced with long-sequence decision tasks. Due to the large state and action space in long-sequence decision-making tasks, the performance of traditional RL significantly decreases when solving such problems. Most studies compensate for this exploration deficiency by designing reward functions [2]. Since desired behaviour is hard to quantify or dense accurate rewards require decomposition into subgoals [3] or achieving progressive objectives, the design of high-performance reward functions is difficult to achieve.

Hierarchical Reinforcement Learning (HRL) improves task success by reducing the task horizon and has been shown to be particularly useful in large action spaces [4]. HRL tackles problems using a divide-and-conquer approach. The high-level policy operates over an abstracted state space and is guided by extrinsic rewards. In contrast, the low-level policy follows an independent strategy and receives separate rewards for successful execution, which are hidden from the global reward.

The superior performance of HRL can be attributed to more effective credit assignment of rewards and more efficient exploration. By abstracting action sequences, HRL can learn the value function more quickly. This is because rewards can be accurately propagated further back in the sequence. According to Driess et al. [5], HRL explores in a semantically meaningful and temporally extended fashion, while traditional RL results in the overexploration of nearby states.

An alternative way to improve RL is through LLM-based guidance. With demonstrated potential in robotics, fault diagnosis, cybersecurity [6] and beyond, LLMs' inference capabilities and extensive general knowledge can significantly improve the performance of RL. Although there is no unified approach to integrating LLMs with RL, various strategies have been explored. LLM is used to preprocess the state, shape the reward, make direct decisions to improve sampling efficiency [7], generate world dynamics, or act as an expert instructors [8,9]. For example, LLM is used to translate the history trajectory [10] or natural language instruction [11] into a task-specific language, thereby improving training efficiency. LLM can act as an implicit rewards [12] or an explicit rewards [13].

In current HRL methods, low-level policies can be predefined by experts or based on prior knowledge. However, HRL's high-level policy is typically learned from scratch, which can limit its performance. This paper proposes a novel approach to leverage LLM priors for assisting high-level policy learning in HRL.

Since LLMs can only process text inputs, this paper first employs a vision-language model (VLM) to convert environmental images into textual descriptions. These descriptions, along with task information, are then fed into the LLM to generate interpretable high-level action instructions (e.g., "pick up hammer" or "go to blue key"). The low-level policy consists of multiple predefined skill modules, which execute corresponding actions based on different high-level instructions. During the training process, LLM does not interact directly with the environment but serves as an extrinsic reward to guide HRL. The weight of LLM guidance will gradually decrease during the training process. HRL transitions from learning from LLM to interactive learning with the environment, thereby exceeding the performance of LLM.

This paper proposes a novel action embedding method for high-level policy. The actions of high-level policy are decomposed into skills and targets. The targets represent the interactive object (for example, a door, key, etc.) selected by the high-level policy, and the interactive object will change as the environment changes. Skills represent the interaction method with the target (for example, basic actions such as pick up, open, etc.). This interaction method will remain consistent in different environments.

In conclusion, the main contributions of LLMHRL are as follows:

- (1) To improve HRL performance in long-sequence tasks, this article proposes a method to train HRL assisted by LLM.
- (2) To address the challenge of high-level policies in long-sequence tasks with large action spaces, this paper introduces a method that decomposes high-level actions into a combination of skills and targets.

2. Related works

This paper proposes a novel method that uses the inference capability and prior knowledge of LLMs to address the challenges of long-sequence decision tasks in sparse reward environments. There are still many problems in applying LLM to long-sequence decision tasks, such as the real-time response of LLM, the inference ability of LLM in complex tasks, and understanding the environment by LLM. Similar research has been conducted to address these issues and formulate practical grounding approaches.

2.1. Real-time and cost-effective implementation with LLMs

When dealing with long-sequence decision tasks, frequent inquiries to LLMs may lead to excessive costs (if a commercial model is used) or longer inference times. In Yu's approach [10], the agent has to consult the LLM for every action, given that the LLM takes several seconds to respond. This approach is incapable of responding in real time. Many approaches have been taken to address this issue. In Bang's approach [11], GPTcache is developed to query semantic responses, increasing the response speed by 2–10 times. Approaches have also been made to limit the use of LLM only during training to train an optimal policy using RL, such as reward design [14], and improving reasoning capabilities [13]. Miao [15] explores the integration of Large Language Models in robotic manipulation by proposing a hierarchical knowledge representation framework that enhances task understanding through multi-source knowledge updates and embedding-based reasoning techniques.

2.2. Improving LLM inference capabilities

LLM's inference capability needs to be improved to address long-sequence tasks directly. As Valmeekam [16] and Lou [17] indicated, LLM instinctively produces a reasoning trace in an autoregressive manner as a hypothetical semantic logic chain. However,

humans possess an internal world model, iteratively exploring options and assessing the likely outcomes. In other words, LLM needs a proper reward mechanism, a world model, and the balance of exploration and exploitation. In Huang's approach [18], a conversation-like inner monologue is proposed to thoroughly observe the environment and use the feedback from the environment. A chain of thought and adding examples is a widely used approach to deconstructing a complex question and provides an interpretable window [19]. In [20], simply adding "let us think step by step" can also improve the zero-shot inference performance of LLM. Another widely used methodology is reinforcement learning with human feedback, such as MaxMin-RLHF [21] with human preference data.

2.3. Alignment input and output with the environment

Aligning input and output with the environment is challenging because LLM needs to perceive environmental information accurately with limited input. In particular, LLM might generate false actions that cannot be executed by the environment, or LLM may not accurately understand the dynamics of the environment, especially implicit interdependencies.

RL is used in the output layer to surpass LLM's alignment deficiency, as the algorithm can learn from specific environments. In Tan's approach [22], actors and critics share the same LLM with low-rank adapters updated by PPO. Carta [23] outputs broad, high-level guidelines and uses PPO to tweak, limiting the misalignment. Szot [24] introduces the MLP layer in the production and splits the action with values for better accuracy. It also introduces embedded task instructions and visual observations by the encoder.

2.4. Comparison with existing LLM-RL frameworks

Table 1
Comparison with existing LLM-RL frameworks.

Method	Model structure	Applicable task sequence length	Inference cost
EUREKA [25]	RL	Short	High
Text2Reward [26]	RL	Short	High
LLMTeach [27]	RL	medium	Low
LLM4PG [28]	RL	medium	Low
SayCan [29]	HRL	long	High
SayCanPay [30]	HRL	long	High
GLIDER [31]	HRL	long	High
LLMHRL (ours)	HRL	long	Low

Table 1 compares the proposed method with existing methods in terms of model structure, applicable task sequence length, and inference cost. Early LLM-RL frameworks use LLMs to design RL reward functions, thereby avoiding complex manual reward design [25,26]. These methods introduce feedback mechanisms to iteratively refine reward designs and improve RL performance. However, the process of refining reward designs requires repeatedly retraining RL agents and evaluating performance, which incurs significant time and computational costs. Subsequently, LLMTeach [27] and LLM4PG [28] propose directly distilling LLM policies into RL, where the LLM provides recommended agent behaviors or trajectories to be stored in the replay buffer, thereby enhancing RL sample efficiency. These methods achieve more direct LLM guidance for RL but face challenges in long-sequence tasks with multiple subtasks, as the limited number of effective trajectories in the replay buffer hinders RL from learning effective policies.

Long-sequence decision tasks are challenging for reinforcement learning because they intensify the exploration-exploitation dilemma. To improve RL performance in long-horizon decision-making tasks, LLM-HRL frameworks employ high-level policies to select predefined skills, thereby reducing the action sequence length and enhancing RL performance. For example, GLIDER [31] fine-tunes an LLM via RL as a high-level policy. SayCan [29] and SayCanPay [30] train evaluation models to assist LLMs in selecting task skills. However, these methods consistently require online LLM inference during the deployment phase, resulting in substantial computational costs and slower model inference speeds, making them difficult to deploy on robotic devices with limited computational capabilities and high real-time requirements.

The proposed LLMHRL method retains the HRL framework and trains an LLM-guided RL agent as the high-level policy. For long-sequence tasks, LLMHRL rapidly improves task success rates with LLM guidance during the early training phase and further enhances its performance through environmental interaction learning in the later stages. Moreover, during deployment, LLMHRL no longer relies on LLM inference; instead, it solely relies on the HRL output for actions, reducing computational and time costs.

Using LLMs to guide RL without altering the RL paradigm is a widely adopted strategy. EUREKA [25] and Text2Reward [26] employ LLMs for dense reward facilitation. However, dense rewards might not be the optimal guidance method, as they only assess immediate student actions. LLMTeach [27] advances this approach by providing recommended actions for clearer guidance, while LLM4PG [28] evaluates student actions in a long-term manner. Hierarchical frameworks reduce the action space and focus on enhancing high-level decision making. Low-level actions are generated by predefined skills. SayCan [29], SayCanPay [30], and GLIDER [31] all directly leverage LLM to make high-level decisions during deployment. HKR [15] uses LLMs to refine a static knowledge graph for high-level decisions. In contrast, LLMHRL employs a dynamic neural network for high-level decisions, incorporating LLM recommendations alongside environmental feedback to enhance robustness.

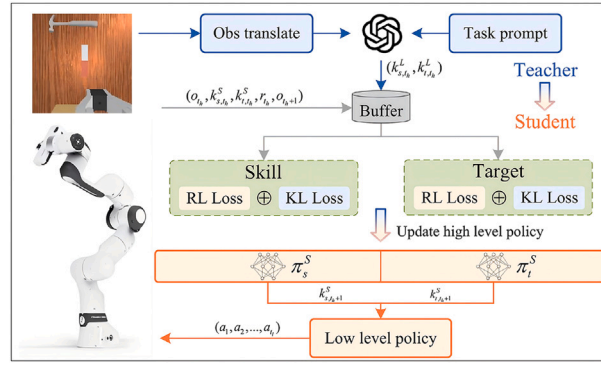


Fig. 1. The LLMHRL framework consists of teacher and student agents. The teacher agent provides recommended task skills and targets as high-level decision suggestions based on textual information from the environment and the current task prompts. The student agent operates within an option-based hierarchical reinforcement learning framework. It selects appropriate task skills and targets based on environmental information as the high-level strategy. The low-level strategy controls the agent to perform specific actions and interact with the environment according to the high-level strategy given by the student agent. During the training process, the high-level policies proposed by both the teacher agent and student agent will be incorporated into the reinforcement learning loss. This integration is achieved by calculating the KL divergence between their policies. This mechanism enables the teacher agent to effectively guide the student agent.

3. Method

This section introduces the research problem and presents the LLMHRL framework.

3.1. Problem formalization

Sequential decision problems are typically modelled as Markov decision processes, represented as $M = (O, A, T, r, \gamma)$. Here, O represents the observation space, A denotes the action space, $T: S \times A \rightarrow P(O)$ represents the transition probabilities, r stands for the reward function and γ represents the discount factor. This Markov decision process aims to learn an optimal policy that maximises $\sum_t \gamma^t r(s_t, a_t)$, where t denotes the time step.

However, reinforcement learning finds it difficult to learn optimal strategies when completing long-sequence decision tasks where rewards are sparse. Using prior knowledge can enhance sample efficiency and accelerate the speed of reinforcement learning training. Option-based hierarchical reinforcement learning divides actions into high-level actions and low-level actions. Each high-level action, or option, comprises a series of low-level actions provided by existing knowledge or policies. The agent learns high-level action policies to complete the entire task.

This paper describes the problem as an option-based hierarchical reinforcement learning approach that introduces temporally extended actions that can span multiple timesteps. An option W is a 3-tuple (L_w, π_w, β_w) representing the states from which the option can be initiated, the option policy, and the option termination function. In a given observation o , a High policy selects an option policy π_w for which $o \in L_w$. Control over actions is delegated to this option for a specific number of timesteps. Once the termination condition β_w is met, the option finishes and cedes control back to the High policy, which then selects a new option to execute.

3.2. Overview

As shown in Fig. 1, the framework consists of teachers and students. For each time step t_h , both the teacher and the student receive observation o_{t_h} . The student will take high-level actions based on o_{t_h} , and low-level policies will generate a series of specific actions $a_1, a_2, \dots, a_{t_h}$ guided by these high-level actions. Once low-level policies reach the termination condition β_w , the observation o_{t_h} will serve as the next observation o_{t_h+1} for high-level decision-making. The teacher will provide a guidance policy based on o_{t_h} . The guidance policy and environmental reward will be used in the high-level policy training of the student agent.

3.3. Teacher

As shown in Fig. 2, this paper defines the sets of high-level options K_s and K_t , where K_s represents the task skills the agent can choose from, and K_t represents the task targets the agent can choose from. The prompt text $p \in P$ obtained by LLM consists of two parts: p_m , which refers to the description of the current task, and p_o , which is the textual description of the current environmental observation o . Translating environmental observations into textual descriptions can be accomplished using existing visual language models such as GPT-4 V and Qwen-VL. Subsequently, LLM will produce a high-level policy a^L based on p . A hardcoded action decomposition module will decompose a^L into k_s^L and k_t^L to guide the student. Here, k_s^L represents the distribution vector of the skill options selected by LLM and k_t^L represents the distribution vector of the target options. Both k_s^L and k_t^L are one-hot encoded. Since LLM does not interact directly with the environment, it cannot confirm whether the generated action a^L is the best high-level decision for the current environmental observation. This paper queries LLM with the prompt text n times and, respectively, records the option probability distributions of k_s^L and k_t^L as π_s^L and π_t^L :

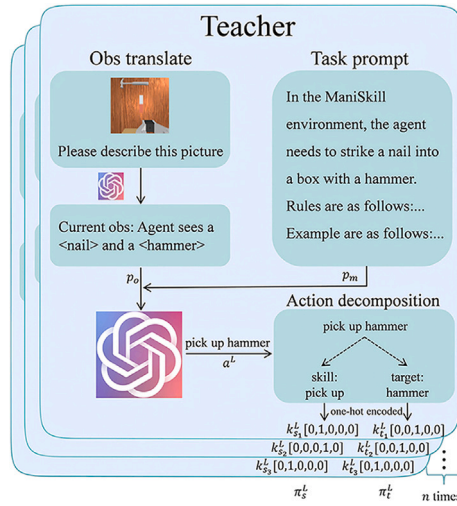


Fig. 2. Teacher pipeline.

$$\pi_s^L = \frac{1}{n} \sum_i^n k_{s_i}^L \quad (1)$$

$$\pi_t^L = \frac{1}{n} \sum_i^n k_{t_i}^L \quad (2)$$

Here, $k_{s_i}^L$ represents the skill option distribution vector obtained by the i -th query to LLM, and $k_{t_i}^L$ represents the target option distribution vector obtained by the i -th query to LLM. π_s^L and π_t^L will provide guidance during the student network's training, which will be explained in the student section.

3.4. Student

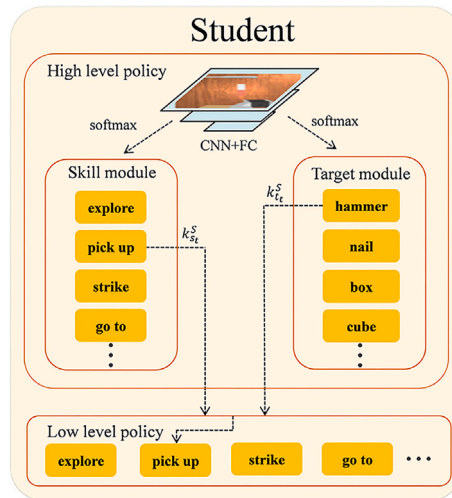


Fig. 3. Student pipeline.

- (1) High-level policy: The student agent adopts an option-based hierarchical reinforcement learning approach, as shown in Fig. 3. For high-level policy, this paper employs a skill module responsible for selecting skill options K_s , and a target module responsible for choosing the target of the current skill action from K_t . The high-level decision comprises a combination of options from these two parts (e.g., “pick up”, “hammer”). This combination method can reduce the action space of high-level decision options, primarily when the student agent handles many skills and has many objects in the environment with which the agent can interact. Specifically, this paper uses two separate, fully connected layers with a standard convolutional layer to perceive the environment. The two fully connected layers use the softmax function as the network output layer, obtaining student

high-level policies π_s^S and π_t^S . The high-level policy obtains option distribution vectors k_s^S and k_t^S through policies π_s^S and π_t^S , respectively, and combines them into a high-level action to be passed into the low-level policy.

- (2) Low-level policy: The low-level policy generates specific low-level actions based on the high-level policy and interacts with the environment. This paper utilises multiple hard-coded policy options based on human expert knowledge. These policy options can also be replaced with pre-trained reinforcement learning modules. When the high-level policy is passed into the corresponding low-level policy option, the low-level policy π_w generates a series of actions $a_1, a_2, \dots, a_{t_l}$ based on the high-level policy's goal and the environment, until the option task reaches a termination condition β_w or exceeds the maximum task steps T_{low} . At the same time, each action produced by the low-level policy will interact with the environment and obtain environmental rewards $r_1, r_2, \dots, r_{t_l}$. The sum of r , from r_1 to r_{t_l} , is considered the reward r_{t_h} obtained by high-level actions. r_{t_h} will be fed into the training process, enabling the student agent to learn from the environment.

3.5. Training module

The high-level policies π_s^S and π_t^S of the student agent are trained by minimizing the following loss function:

$$L(s, t) = L_{RL}(s) + L_{RL}(t) + \lambda \left(\mathbb{E}_{o \sim \pi_s} \mathcal{H}(\pi_s^L(o) \| \pi_s^S(o)) + \mathbb{E}_{o \sim \pi_t} \mathcal{H}(\pi_t^L(o) \| \pi_t^S(o)) \right) \quad (3)$$

$$\mathcal{H}(\pi_t^L(o) \| \pi_t^S(o)) = D_{KL}(\pi_t^L(o) \| \pi_t^S(o)) = \pi_t^L(o) \log \frac{\pi_t^L(o)}{\pi_t^S(o)} \quad (4)$$

$$\mathcal{H}(\pi_s^L(o) \| \pi_s^S(o)) = D_{KL}(\pi_s^L(o) \| \pi_s^S(o)) = \pi_s^L(o) \log \frac{\pi_s^L(o)}{\pi_s^S(o)} \quad (5)$$

$L_{RL}(s)$ and $L_{RL}(t)$ represent the loss functions used for π_s^S and π_t^S , respectively. Adopting standard reinforcement learning loss functions, this paper uses the Proximal Policy Optimization (PPO) algorithm as the reinforcement learning algorithm. It achieves learning from the environment by maximising the expected return of the agent. $\mathcal{H}(\pi_s^L(o) \| \pi_s^S(o))$ and $\mathcal{H}(\pi_t^L(o) \| \pi_t^S(o))$ represent the difference between the high-level policy recommended by the teacher and the high-level policy provided by the student. This policy difference can be calculated using the Kullback-Leibler (KL) divergence or cosine similarity. This paper introduces an annealing parameter λ to control the degree of guidance from the teacher agent to the student agent. Expressly, the annealing parameter λ is set based on the following formula:

$$\lambda_i = \begin{cases} \lambda_0 - \frac{\lambda_0}{i_1} * i & \text{if } i < i_1 \\ 0 & \text{else} \end{cases} \quad (6)$$

$$\lambda_0 = \frac{L_{RL}(s) + L_{RL}(t)}{\left(\mathbb{E}_{o \sim \pi_s} \mathcal{H}(\pi_s^L(o) \| \pi_s^S(o)) + \mathbb{E}_{o \sim \pi_t} \mathcal{H}(\pi_t^L(o) \| \pi_t^S(o)) \right)} \quad (7)$$

At the beginning of training, to balance the KL divergence loss and the RL loss and avoid a large gap, this paper uses (7) to set λ_0 . As the training steps i increase, λ will linearly decrease with a decay rate $\frac{\lambda_0}{i_1}$ and become 0 after i_1 training steps. At this point, the student agent will learn entirely through interactions with the environment, further enhancing its robustness for specific tasks. The entire learning process is summarized in Algorithm 1.

4. Experiment

In this section, LLMHRL is evaluated through various experiments and the experimental results demonstrate that LLMHRL has significant advantages over other baseline methods.

4.1. Environment

Minigrid) MiniGrid provides a grid-based visual environment where users can freely set the grid size and interactive objects for the agent, such as walls, doors, keys, etc. By interacting with objects, the state of an object can be changed, such as a closed door being turned into an open door. The agent navigates to specific objects and completes a series of interactions to finish tasks such as 'put all the balls into the box.' The agent's action space is discrete and includes move forward, turn left, turn right, open, close, stop. The observation space is set as a locally observable environment of size 3x3 and has built-in memory functionality that can record the block or target that has already been discovered.

ManiSkill) ManiSkill3 is a GPU-parallelised robotics simulation platform specifically designed to accelerate the development of generalisable manipulation skills for embodied AI agents. It provides a scalable framework for training and evaluating various contact-rich manipulation tasks, including dexterous object reorientation, tool-based drawing, mobile manipulation, and humanoid control in both artist-designed scenes and real-world digital twins. The platform enables detailed configuration of robotic agents, supporting heterogeneous robot morphologies, multi-modal visual inputs (e.g., point clouds, voxels), and customisable physical properties.

Algorithm 1 Student learning process.

```

Initialize s and t, the parameters of the policy  $\pi_s^S$  and  $\pi_t^S$ . Pre-train the
low-level policy options  $\{\pi_{\omega}\}$ .
for  $i = 1, 2 \dots T$  do
  Set data buffer  $D = \{\}$ 
  for  $j = 1$  to batch size, do
    for  $t_h = 1$  to  $T_{high}$  do
      Set  $\beta = False$ ,  $t_l = 0$ ,
       $k_{s,t_h}^S = \pi_s^S(o_{t_h})$ 
       $k_{t,t_h}^S = \pi_t^S(o_{t_h})$ 
      while  $t_l < T_{low}$  do
         $a_{t_l}, \beta = \pi_{\omega}(o_{t_l}, k_{s,t_h}^S, k_{t,t_h}^S)$ 
        Execute action  $a_{t_l}$ , observe  $r_{t_l}, o_{t_h+t_l+1}$ 
         $t_l += 1$ 
        if  $\beta == True$  then
           $r_{t_h} = \sum_{i=0}^{t_l} r_i$ 
           $o_{t_h+1} = o_{t_l}$ 
          Break
        end if
      end while
      Store  $(o_{t_h}, k_{s,t_h}^S, k_{t,t_h}^S, r_{t_h}, o_{t_h+1})$  into buffer  $D$ 
    end for
  end for
  for each transition  $(o_{t_h}, k_{s,t_h}^S, k_{t,t_h}^S, r_{t_h}, o_{t_h+1}) \in D$  do
    generate a textual description p for the teacher agent based on the
    environment  $o_{t_h}$ 
    Obtain the teacher's guiding actions  $\pi_s^L, \pi_t^L$  based on (1) and (2)
  end for
  for each gradient descent step, do the following: do

```

$$L(s, t) = L_{RL}(s) + L_{RL}(t) + \lambda_i \left(\mathbb{E}_{o \sim \pi_s} \mathcal{H}(\pi_s^L(o) \| \pi_s^S(o)) + \right. \\ \left. \mathbb{E}_{o \sim \pi_t} \mathcal{H}(\pi_t^L(o) \| \pi_t^S(o)) \right)$$

```

    end for
  end for

```

4.2. Baselines

In the experiments, this paper evaluates LLMHRL using four baseline methods.

LLMTeach [27]: LLMTeach is an end-to-end reinforcement learning approach guided by a large language model. The model learns task strategies based on the behavioural suggestions given by LLM. Similarly to LLMHRL, this method also uses LLM for strategy guidance, so this paper uses LLMTeach as a comparative experiment.

LLM: LLM uses LLM for high-level decision-making, giving high-level actions that the agent should take. The low-level policy module gives specific actions that the agent needs to perform. With this approach, the LLM must remain online to make real-time decisions. Compared with this method, it can be evaluated whether LLMHRL effectively learns high-level policies from LLM in the early stages of training and gradually surpasses LLM in the later stages of training.

Inner Monologue [18]: Inner Monologue adopts scripted modules to provide language feedback in the form of object recognition, success detection, and description of the progress of the task. LLM will review its high-level decisions from language feedback. With this approach, the LLM also must remain online to make real-time decisions. In comparison with this method, it can be evaluated whether LLMHRL could surpass LLM's chain-of-thought capability.

Hierarchical RL [32]: Hierarchical RL uses an option-based hierarchical reinforcement learning method. Each option represents a low-level policy consistent with LLMHRL. This paper compares Hierarchical RL with LLMHRL to assess whether LLMHRL effectively learns the high-level policies of the LLM.

RL (PPO) [33]: RL (PPO) is an end-to-end reinforcement learning baseline method that uses the Proximal Policy Optimization algorithm. This approach demonstrates the performance of a typical reinforcement learning algorithm in environments with sparse rewards.

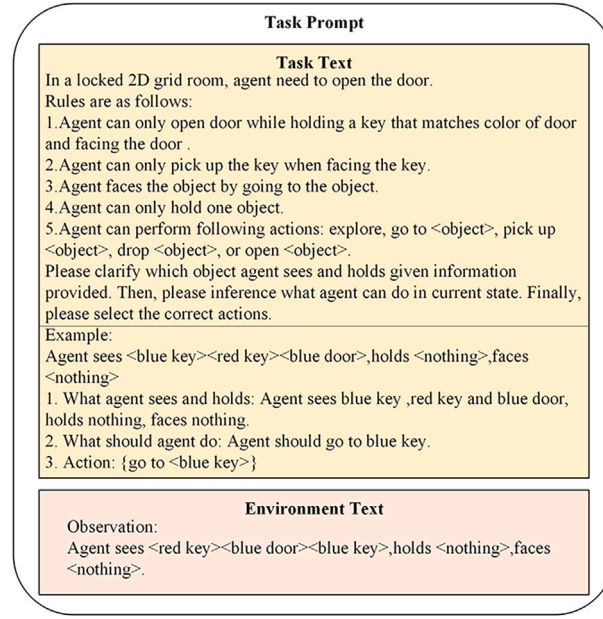


Fig. 4. The examples of task prompt.

4.3. Evaluation metrics

To evaluate the robustness of each method, this paper defines three key performance indicators.

- (1) **Success Rate:** The success rate measures the percentage of episodes in which the agent reaches its final target successfully. It directly reflects the reliability and effectiveness of the learned policy.
- (2) **Average Return:** The average return represents the mean of the cumulative rewards that the agent receives per episode. A higher average return generally indicates a more optimal policy, as the agent maximises its rewards over time.
- (3) **Convergence Speed:** Convergence speed measures the algorithm's sample efficiency by quantifying the number of training steps required to learn a policy that achieves at least 80 % success rate, demonstrating how quickly the algorithm can develop a competent strategy.

4.4. Large language model

This paper refers to Chat-GPT4 via the api as LLM to guide the high-level policy of LLMHRL. Its powerful inference capabilities can provide practical guidance strategies for LLMHRL. The LLM method in comparative experiments also utilises this model to ensure fairness.

To improve the inference capabilities of LLM, this paper uses chain-of-thought style text prompts, including task text and environment text. The task text provides the rules for the current task and specific examples, helping LLM understand the meaning of each high-level decision and the format of the output text. The environment text is derived from the agent's observations, analyzed and interpreted by GPT-4 V. The LLM needs to provide appropriate high-level policy guidance based on different environment observation texts. The high-level decisions output by the LLM include the explanation of the current environment, the subsequent high-level decision that the agent should take, and the keywords for the corresponding decisions. A specific example is shown in Fig. 4.

4.5. Experiments on MiniGrid

This section evaluates the performance of LLMHRL across four MiniGrid tasks of escalating difficulty, comparing it against other baseline methods. Experimental results demonstrate that LLMHRL consistently achieves a higher success rate, better average return, and significantly faster training convergence, particularly on more complex tasks where traditional RL methods completely fail. Furthermore, ablation studies confirm the effectiveness of LLMHRL's design; decomposing high-level policies into separate 'skills' and 'targets' is shown to be crucial for improving performance and exploration efficiency by reducing the action space. Moreover, ablation studies analyze the impact of key learning parameters on the success rate of LLMHRL.

4.5.1. Experimental setting

- (1) **Task Configuration:** Four distinct tasks have been built with the MiniGrid framework with ascending difficulty, as shown in Fig. 5. Specifically, the more challenging task requires a longer decision chain and is less tolerant of incorrect behaviour to obtain a reward and reach the target.

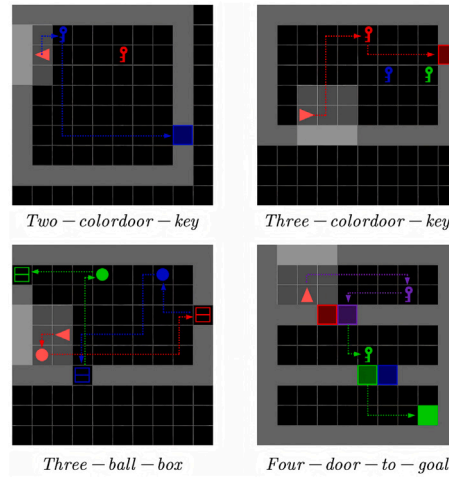


Fig. 5. Four types of task examples, with ascending difficulty. Two-colordoor-key and three-colordoor-key are single key-door pairing problem with distractor. Three-ball-box are multiple sorting problem. Four-door-to-goal are multi-layer exploring and key-door pairing problem. The dashed line shows the optimal trajectory obtained by the agent using different skills and targets.

Two-colordoor-key (easy): After generating two keys and one door, the agent needs to find and retrieve the right colour key for the door and use it to open the door.

Three-colordoor-key (medium): After generating three keys and one door, the agent needs to find and retrieve the correct colour key for the door and use it to open it.

Three-ball-box (medium): Having generated three balls and three boxes, the agent needs to find and place the balls in the correct coloured boxes.

Four-door-to-goal (hard): Creating three levels, the agent needs to explore and locate the correct key for the corresponding door, retrieve the key, open the door, and drop the key in the first and second levels. The environment will also generate a fake door with no key to open as interference. In the final level, the agent must find and reach the target.

(2) Reward Function:

The above tasks use the same reward function:

$$r = (1 - \text{step}_{\text{finish}} / \text{step}_{\text{max}}) * \text{done} \quad (8)$$

$\text{step}_{\text{finish}}$ represents the number of low-level policy steps taken by the agent to complete the current task. step_{max} represents the maximum number of low-level policy steps that the agent can execute. done means the reward is only given after finishing. Such a reward function design ensures effective task completion despite sparse reward conditions.

(3) Low level Policy:

In MiniGrid environment, this paper builds a low level policy library with skills and targets, featuring five different skills and fifteen different targets, as shown in Table 3.

Table 2
Minigrid skills and their corresponding low level policies.

Skill	Low level policy
explore	A*
go to	A*
pick up	one step action
put down	one step action
open	one step action

Table 3
Minigrid tasks with available skills and targets.

Task	Available Skill	Available Target
Two-colordoor-key	all	red/blue key, blue door
Three-colordoor-key	all	red/blue/green key, blue door
Three-ball-box	all	red/blue/green ball, red/blue/green box
Four-door-to-goal	all	purple/green key, red/blue/green/purple door, target

This paper adopts the A* algorithm for path finding low-level policies ‘explore’ and ‘go to’. ‘explore’ randomly selects one nearby empty space as the destination, and ‘go to’ extracts the target coordinate as the destination. For other skills like ‘pick up’, ‘put down’, ‘open’, this paper uses actions provided by Minigrid environment. The details are listed in Tables 2 and 3.

4.5.2. Competitive results

Table 4 and Fig. 6 show the success rate performance of the five methods deployed in four environmental tasks. LLMHRL achieved the highest success rate in three of the four tasks. This demonstrates that LLMHRL can effectively adapt to different tasks. At the same time, Table 6 indicates that LLMHRL converges significantly faster than other methods. This indicates that LLMHRL fully utilised the

Table 4
The success rate of four tasks.

Success Rate	Two-colordoor-key	Three-colordoor-key	Three-ball-box	Four-door-to-goal
LLMHRL	0.928	0.883	0.936	0.887
LLMTech	0.928	0.903	0.813	0.405
Inner Monologue	0.890	0.800	0.780	0.710
LLM	0.770	0.590	0.600	0.480
RL (Hierarchy)	0.216	0.048	0.000	0.002
RL (PPO)	0.064	0.017	0.000	0.000

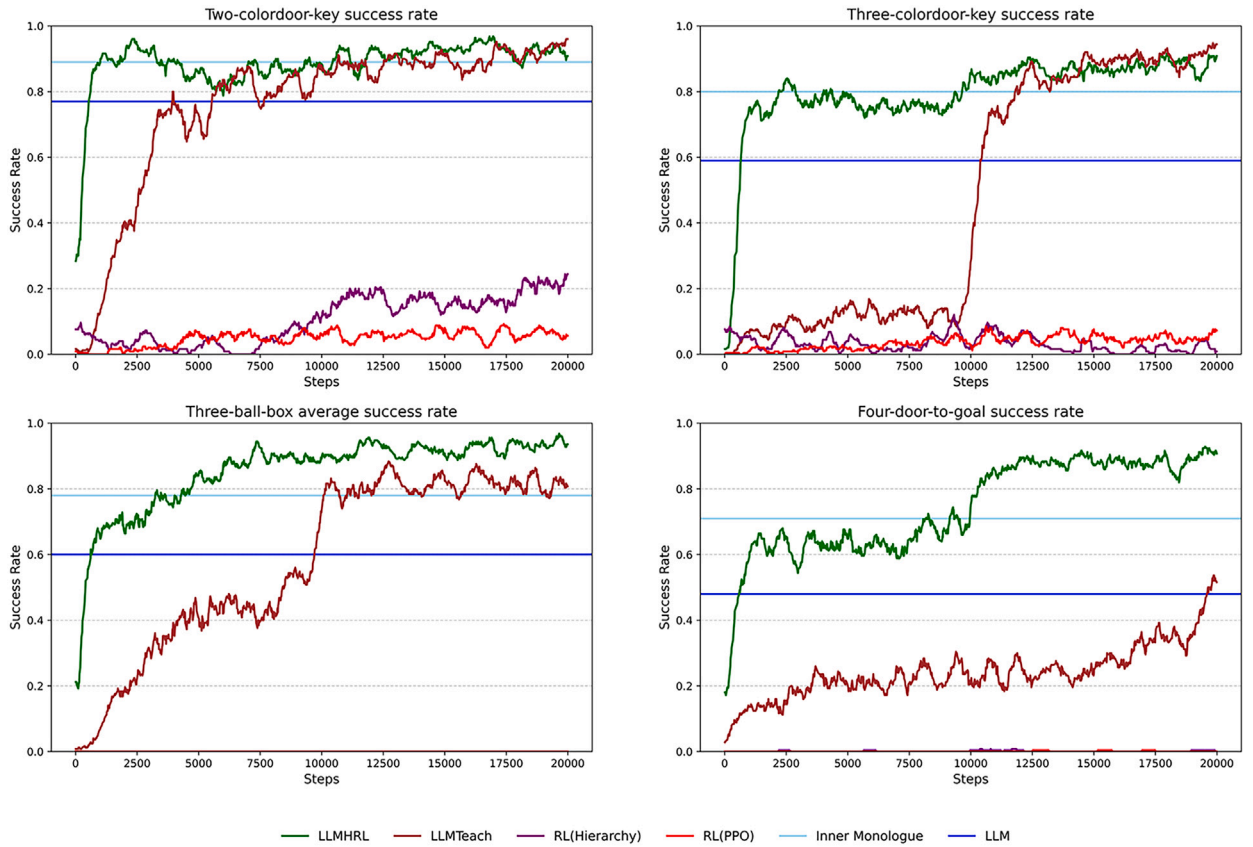


Fig. 6. Success rate of four tasks during training.

Table 5
The average return of four tasks.

Average Return	Two-colordoor-key	Three-colordoor-key	Three-ball-box	Four-door-to-goal
LLMHRL	0.726	0.668	0.591	0.609
LLMTech	0.759	0.667	0.528	0.243
Inner Monologue	0.700	0.600	0.450	0.370
LLM	0.600	0.520	0.320	0.190
RL (Hierarchy)	0.087	0.024	0.000	0.000
RL (PPO)	0.027	0.009	0.000	−0.001

Table 6
The convergence speed of four tasks.

Convergence Speed	Two-colordoor-key	Three-colordoor-key	Three-ball-box	Four-door-to-goal
LLMHRL	1250	2000	5000	11,500
LLMTeach	5000	10,000	11,000	∞
Inner Monologue	0	0	∞	∞
LLM	∞	∞	∞	∞
RL (Hierarchy)	∞	∞	∞	∞
RL (PPO)	∞	∞	∞	∞

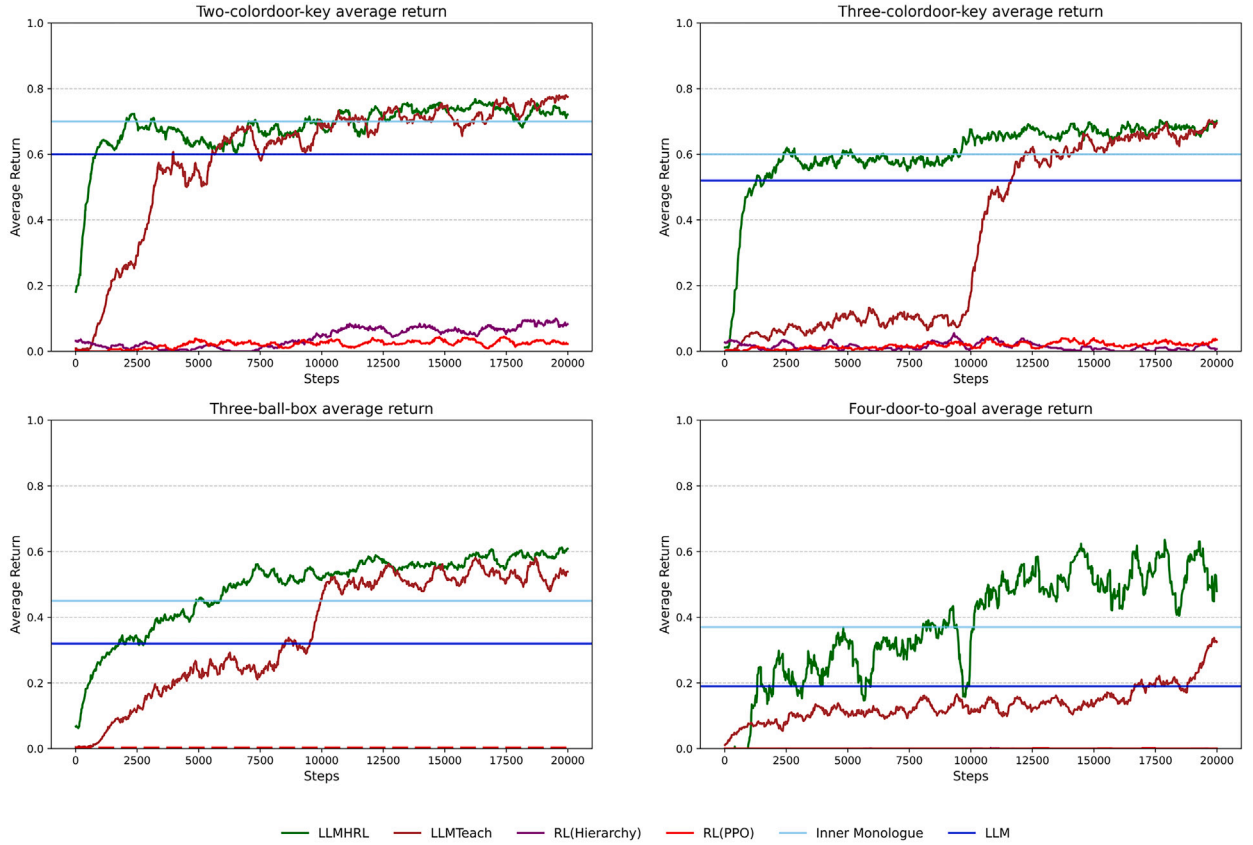


Fig. 7. Average return of four tasks during training.

LLM guidance in the early stages of training and quickly improved task performance. Table 5 and Fig. 7 present the average return performance of the five methods deployed in four environmental tasks. In particular, LLMHRL secured the highest average rewards in three of these tasks. Specifically, the average reward of LLMHRL exceeds that of LLM and Inner Monologue. This demonstrates that such a teacher-student paradigm results in gradual policy learning, which is more robust than introducing environment feedback to LLM. Furthermore, as the difficulty of the tasks increases, LLMHRL can still maintain a high success rate and average return, while the performance decline of the other methods is more pronounced.

Although LLMTeach performs well on most tasks and converges to an effective policy in 10,000 steps, it struggles with more complex ones. This issue stems from the student agent learning low-level actions, which demand both the mastery of various skills and their execution in a precise sequence. As a result, when the decision chain of a task is longer, the time required for the agent to learn the correct policy with LLM guidance increases significantly.

Conventional RL methods, regardless of whether a hierarchical architecture is used, are unable to learn effective policies. The introduction of a hierarchical structure to reduce the decision-chain length proves to be insufficient to yield satisfactory performance. The results demonstrate that LLM integration is essential to enable the agent to overcome these learning challenges.

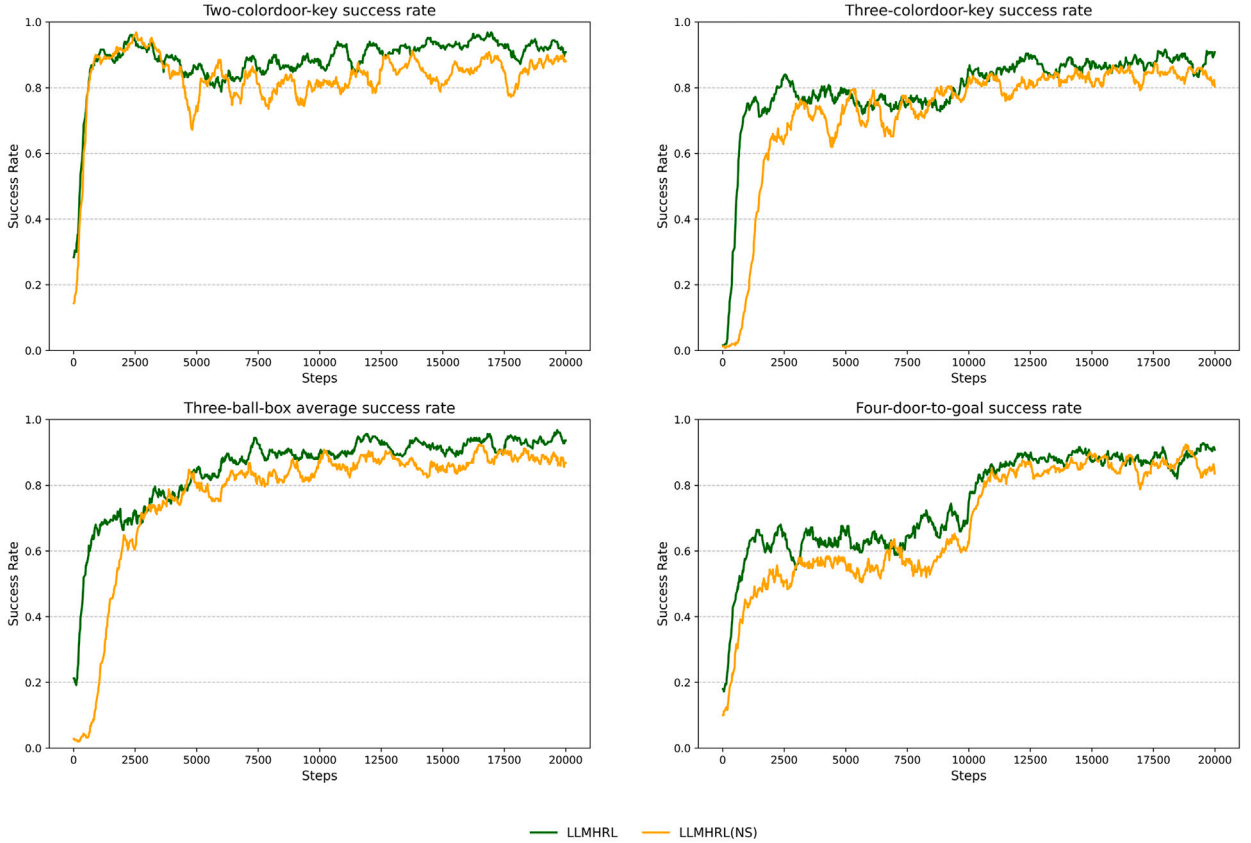
4.5.3. Ablation study on the decomposition of skills and targets

LLMHRL decomposes the high-level policy into a combination of skills and targets. Whether this decomposition-combination approach is effective requires experimental validation. Therefore, this paper conducts a set of ablation experiments. LLMHRL is

Table 7

The success rate and average return of ablation study.

Success Rate	Two-colordoor-key	Three-colordoor-key	Three-ball-box	Four-door-to-goal
LLMHRL	0.928	0.883	0.936	0.887
LLMHRL (NS)	0.874	0.842	0.884	0.867
Average Return	Two-colordoor-key	Three-colordoor-key	Three-ball-box	Four-door-to-goal
LLMHRL	0.726	0.681	0.591	0.518
LLMHRL (NS)	0.698	0.596	0.507	0.609

**Fig. 8.** Success rate of four tasks during training of ablation study.

compared to the method in which all high-level strategy options are given directly without decomposition, referred to as LLMHRL (NS).

As shown in Table 7, Fig. 8, and Fig. 9, decomposing the action space into skills and targets improves performance in four tasks. The reason behind this is that challenging tasks expand the action space, leading to a decrease in exploration efficiency. By splitting the action space into two parts, LLMHRL can effectively reduce the action space. This allows the high-level policy to better learn the LLM guidance policy.

4.5.4. Ablation study of the learning parameters i_1 and λ_0

LLMHRL adopts an annealing parameter λ to control the degree of guidance from the teacher agent to the student agent. Specifically, it starts from an initial value λ_0 and decreases to 0 in the i_1 training steps. Whether this guidance is effective and how the initial value λ_0 and the termination steps i_1 affect the training requires experimental validation. This section ablates λ_0 and i_1 .

As shown in Fig. 10, λ_0 affects the speed of convergence. In the original Formula 7, λ_0 is calculated by balancing teacher guidance and environment reward in a 1:1 ratio. Increasing or decreasing either factor reduces training efficiency, as shown in the results with $2\lambda_0$ and $0.5\lambda_0$. Moreover, $\lambda_0 = 0$ without teacher guidance cannot facilitate training.

The termination steps i_1 determine whether the training can succeed or not. If i_1 is set too small, such as terminating at 1000 steps, training cannot succeed. 2000 steps are optimal, but increasing the steps only harms the performance while still allowing for success. This result remains consistent when λ_0 increases or decreases.

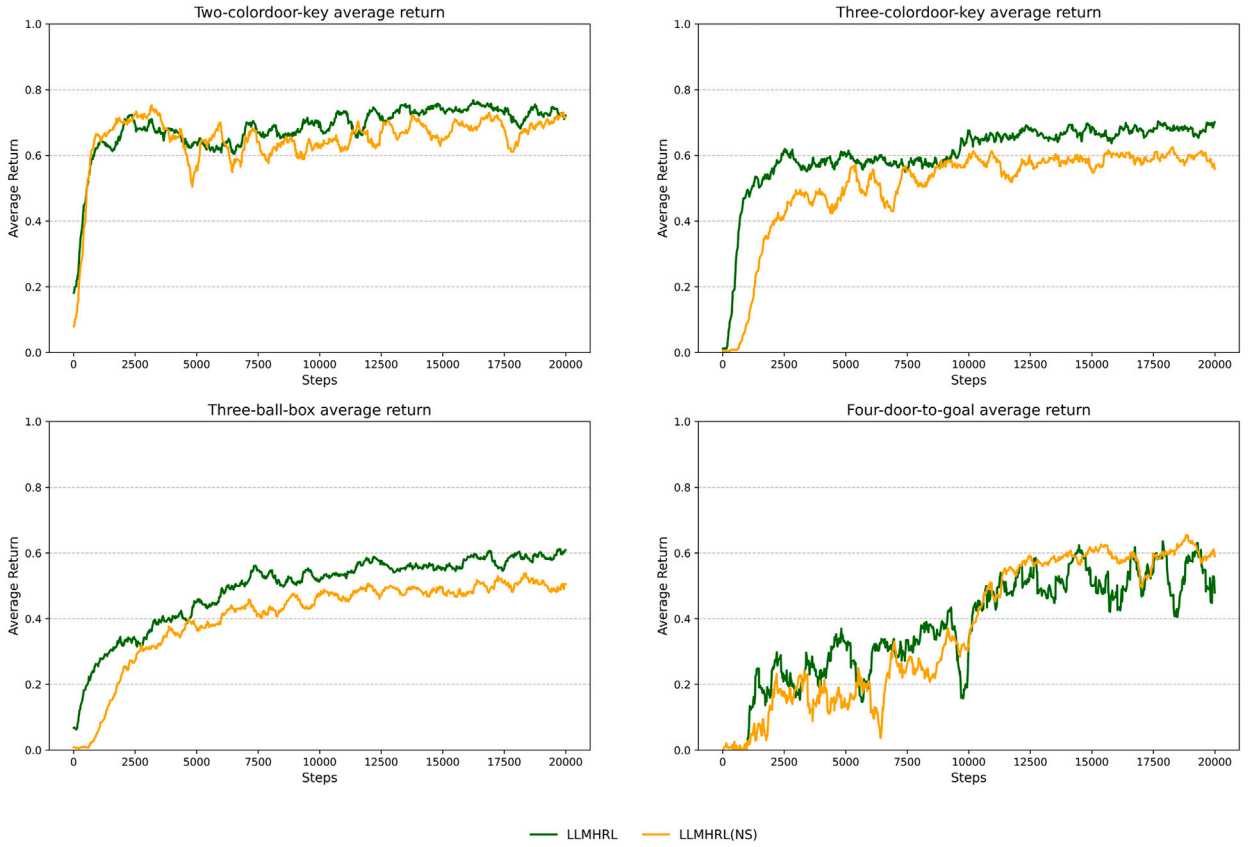


Fig. 9. Average return of four tasks during training of ablation study.

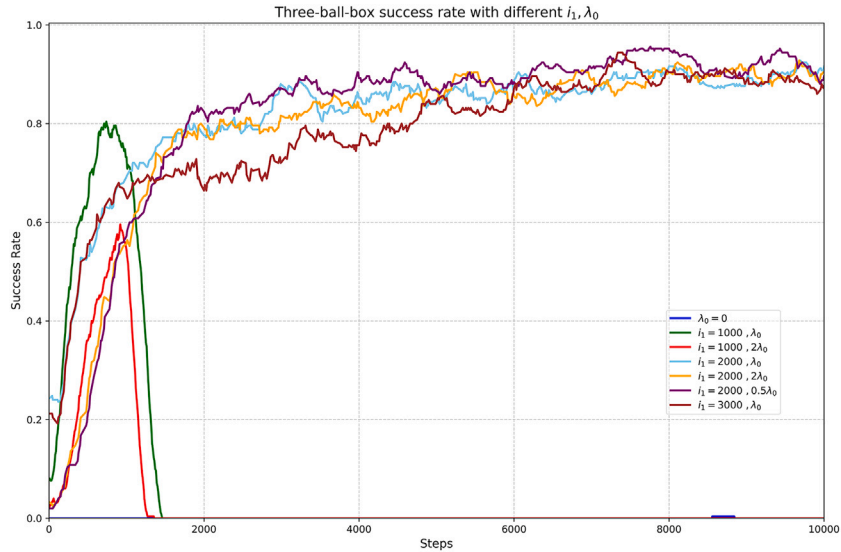


Fig. 10. Three-ball-box success rate with different i_1 and λ_0 .

4.6. Experiments on ManiSkill

This section presents three progressively challenging tabletop manipulation tasks on the ManiSkill platform: a Single Cube Pick-Up, a Triple Cube Pick-Up, and a Hammer Strike Nail task—where the robot, guided by partial visual observations and a pre-configured translator that converts RGB images into natural language, must execute long-sequence decision-making under varying skill and

target options. Results indicate that LLMHRL not only achieves the highest success rates and average returns across all tasks but also demonstrates rapid learning progress, overcoming challenges such as LLM's inability to incorporate execution feedback and RL's struggles in sparse reward settings.

4.6.1. Experimental setting

- (1) **Task Configuration:** In ManiSkill, this paper designs three robotic tabletop manipulation tasks. These tasks are long-sequence decision-making tasks with gradually increasing difficulty.

For the single-cube pick-up task, this paper configures a cube with a randomised position and colour on the table alongside three differently coloured targets, as shown in Fig. 11. The robot must pick up the cube and place it on a target matching its colour.

The Triple Cube Pick-Up task introduces three distinctively coloured cubes and corresponding targets with randomised positions, as shown in Fig. 12. The robot must pick up each cube and place it on the correspondingly coloured target. This configuration significantly extends the decision-making horizon by requiring sequential manipulation of multiple objects.

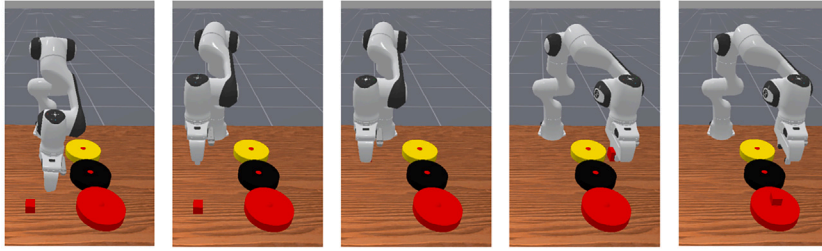


Fig. 11. The examples of single cube pick up task.

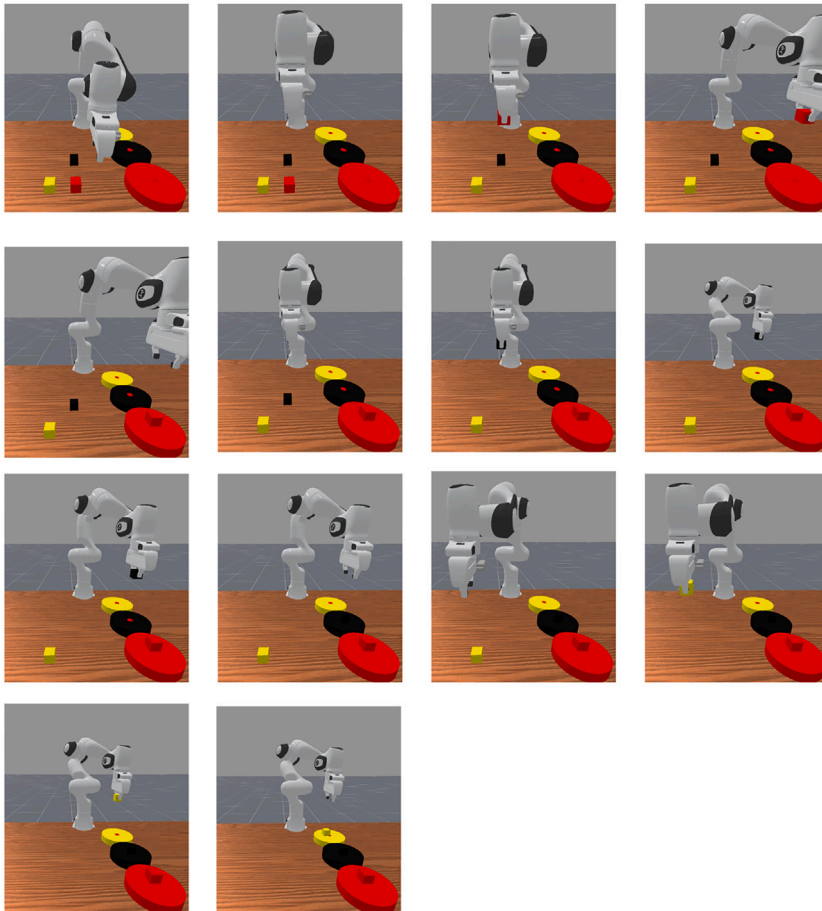


Fig. 12. The examples of triple cube pick up task.

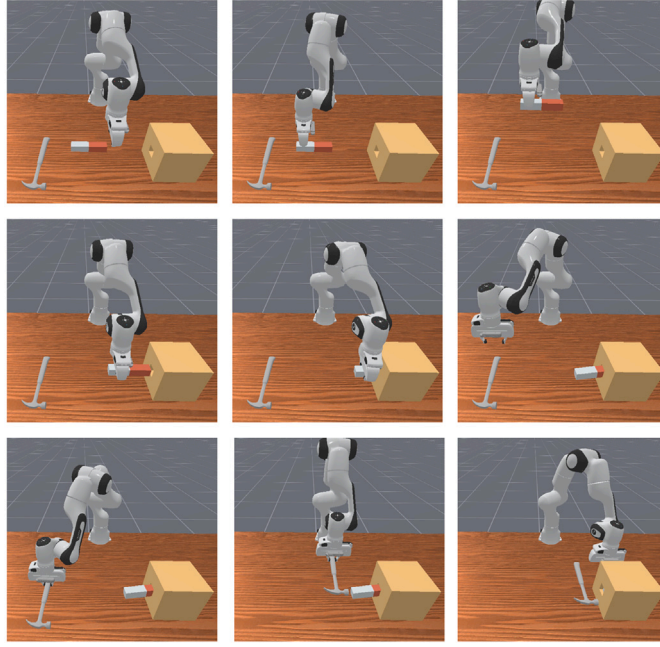


Fig. 13. The examples of hammer strike nail task.

The Hammer Strike Nail task presents greater challenges with a hammer, nail, and a cube that contains a hole, as shown in Fig. 13. The robot must first grasp and align the nail with the hole before inserting it. Subsequently, it needs to grasp the hammer, align it with the nailed position, and strike to complete the task. This design introduces complex task rules and strict action-sequencing requirements, further increasing difficulty.

(2) **Reward Function:** The above tasks use the same reward function:

$$r = (2 - 0.5 * (steps_{finish} / steps_{max})) * done \quad (9)$$

(3) **Low level Policy:**

Across all tasks, the high-level policy only receives partial visual observations through RGB images captured above its end-effector. This paper adopts six low-level policy options: explore, go to, pick up, place, insert, and strike, as shown in Tables 8 and 9. The low-level policy is driven by RRT algorithm, which plans the trajectory τ for the robot starting from the initial point x_0 , and passing through the target point x_i . For each skill and target combination, the target point is previously annotated by a human and extracted from the environment.

Table 8
Maniskill Skills and their corresponding Low Level Policies.

Skill	Low level policy
explore	RRT*
go to	RRT*
pick up	close gripper
place	open gripper
strike	RRT*
insert	RRT*

Table 9
Maniskill tasks with available skills and targets.

Task	Available Skill	Available Target
Single Cube Pick Up	all	red/yellow/black target, red cube
Triple Cube Pick Up	all	red/yellow/black target, red/yellow/black cube
Hammer Strike Nail	all	hammer, nail, hole

4.6.2. Competitive results

Fig. 14 and Table 10 show the success rate of four methods during training. Across three progressively challenging tasks, LLMHRL achieves success rates of 92 %, 75 %, and 72 %, consistently outperforming standalone LLM. Fig. 15 and Table 11 show the average return of the four methods during training. Performance trends align with those observed in the success rate curves. As shown in Table 12, LLMHRL demonstrates a rapid growth in the success rate guided by LLM.

Inner Monologue fails to succeed in some scenarios despite receiving feedback from the language environment. This limitation stems from the LLM's limited ability to make task-specific adjustments. Without such feedback, the LLM itself performs worse than

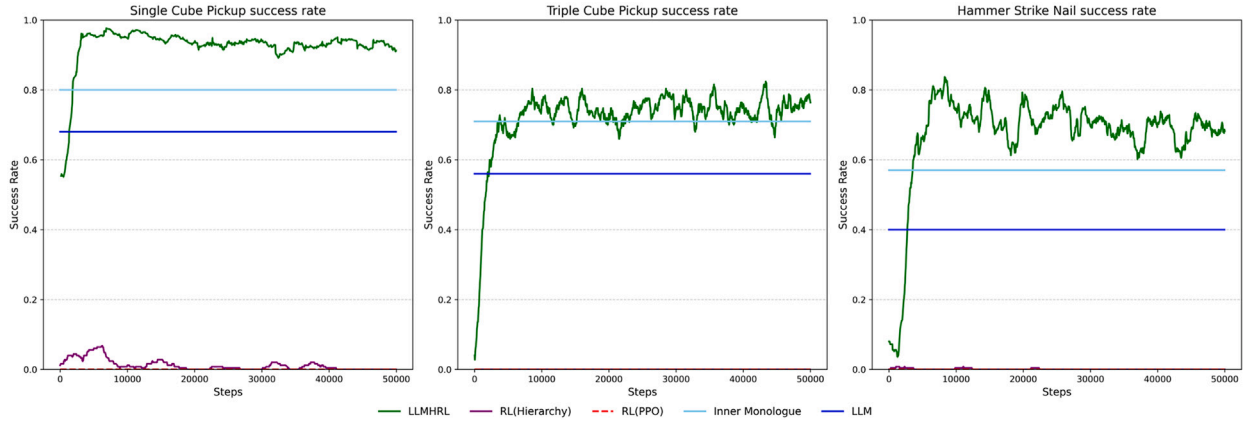


Fig. 14. Success rate of four tasks during training.

Table 10
The success rate of three tasks.

Success Rate	Single Cube Pick Up	Triple Cube Pick Up	Hammer Strike Nail
LLMHRL	0.95	0.77	0.71
Inner Monologue	0.80	0.71	0.57
LLM	0.68	0.56	0.45
RL (Hierarchy)	0.01	0.00	0.01
RL (PPO)	0.00	0.00	0.00

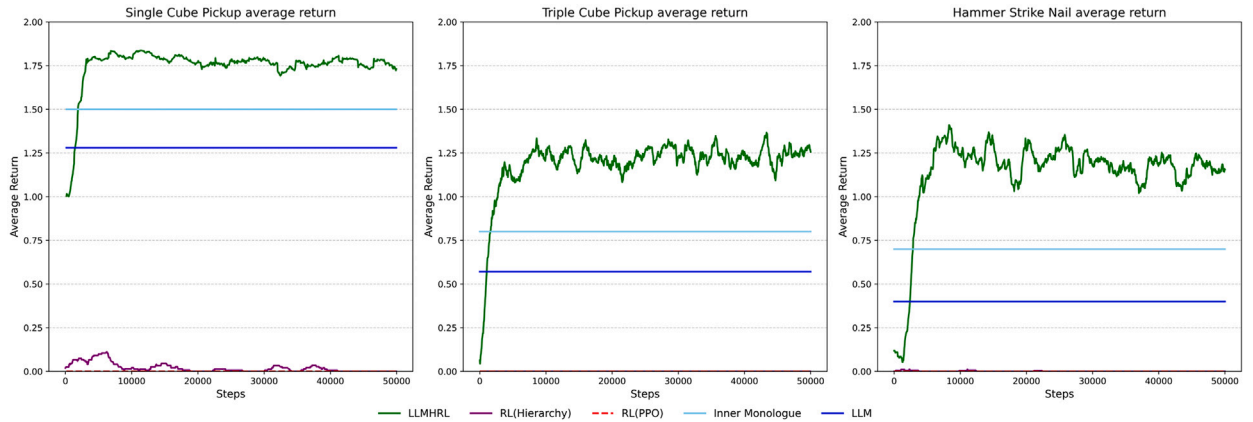


Fig. 15. Average return of four tasks during training.

Table 11
The average return of three tasks.

Average Return	Single Cube Pick Up	Triple Cube Pick Up	Hammer Strike Nail
LLMHRL	1.76	1.25	1.16
Inner Monologue	1.50	0.80	0.70
LLM	1.28	0.57	0.40
RL (Hierarchy)	0.18	0.00	0.02
RL (PPO)	0.00	0.00	0.00

Table 12
The convergence speed of three tasks.

Convergence Speed	Single Cube Pick Up	Triple Cube Pick Up	Hammer Strike Nail
LLMHRL	5000	10,000	10,000
Inner Monologue	0	∞	∞
LLM	∞	∞	∞
RL (Hierarchy)	∞	∞	∞
RL (PPO)	∞	∞	∞

Inner Monologue. Meanwhile, HRL occasionally completes tasks during the early training phases. However, the sparse reward setting prevents HRL from learning effective policies. PPO, which directly outputs low-level actions, maintains a 0 % success rate throughout training in the absence of dense reward signals.

4.7. Evaluation on the physical robot

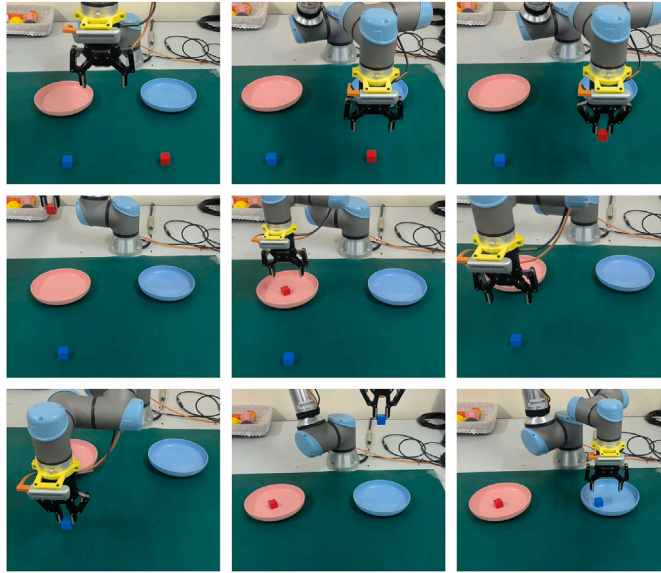


Fig. 16. Average return of four tasks during training.

This paper evaluates the performance of LLMHRL on a physical robot, as shown in Fig. 16. The evaluation platform employs a UR5 robotic arm with six degrees of freedom, equipped with a Robotiq-85 gripper at its end effector to provide additional grasping capabilities. An Intel RealSense D435 RGB-D camera is mounted on the end effector to capture visual inputs. The testing task resembles the “pick up cube” task in the ManiSkill environment, requiring the robot to place a red cube and a blue cube into correspondingly coloured bins on a table.

First, this paper configures a scenario in the ManiSkill environment identical to the real-world setup and trains LLMHRL in this simulated setting. The trained model is then transferred to the physical robot for testing. During the evaluation, RGB images from the camera are fed as state input to the model. The model outputs high-level actions that are executed through predefined robotic skills. To validate the performance of the physical robot, this paper conducts 20 episodes in real-world scenarios, achieving 16 successful completions. The results demonstrate that LLMHRL can be reliably deployed on physical robots in real-world environments.

4.8. LLM failure analysis

Table 13
Task failure categories.

	LLM Success	Observation Abstraction Failure	LLM Formatting Failure	LLM Reasoning Failure	Low-level Execution Failure
Four-door-to-goal @Minigrid	48 %	15 %	10 %	27 %	0 %
Hammer Strike Nail @Maniskill	45 %	10 %	5 %	35 %	5 %
Pick Up cube @Real-world Robot	40 %	20 %	2 %	18 %	20 %

To investigate the deficiencies of LLMs used as zero-shot, high-level policies, this paper conducts a detailed failure analysis. The failures are categorised by human judges into four different types. As shown in Table 13, the overall success rate is limited, primarily because LLMs lack inherent task-specific knowledge and their outputs can be inconsistent.

Observation Abstraction Failure: This situation occurs when using a VLM to convert environmental states into text descriptions, resulting in ambiguous or confusing representations for the LLM. For example, an input like ‘Sees red cube and green cube, Holds green cube’ might confuse the LLM into redundantly outputting ‘Pick up green cube.’

LLM Formatting Failure: This happens when the LLM’s output does not conform to the expected format that the low-level modules can execute. For instance, when required to output a predefined action, the model might generate ‘exploit’ instead of the valid command ‘explore’ causing a parsing error.

LLM Reasoning Failure: This category covers instances where the LLM does not understand the logic of the environment or the task objective. For example, when presented with a ‘red key’ and a ‘blue door,’ an LLM might default to outputting ‘pick up red key’ due to a simple object-association bias, rather than reasoning that it needs to find a blue key. The correct action would be to ‘explore’ for the appropriate key.

Low-level Execution Failure: This type of failure arises when a previous action makes a subsequent command physically impossible to execute. For example, after the LLM instructs the robot to place one cube on top of another, the robot might not be able to execute a later command to pick up the bottom cube, causing the task to fail.

The distribution of failure types varies significantly depending on the task and environment. The four-door-to-goal task, which requires frequent exploration, resulted in a higher incidence of LLM Formatting Failures, as the increased number of commands provided more opportunities for generating malformed output. In contrast, the Hammer-strike-Nail environment, which requires understanding the procedural dependency of placing the nail before striking it, predominantly triggered LLM Reasoning Failures. It is projected that Low-level Execution Failures would occur far more frequently in complex real-world environments where unpredictable physical interactions are common.

4.9. Computational cost and hardware configuration

Table 14
Latency in one environment interaction.

	Training	Deployment
LLMHRL	250 ms	<10 ms
LLMTeach	5000 ms	<10 ms
Inner Monologue	No training	25,000 ms
LLM	No training	5000 ms
RL (Hierarchy)	<10 ms	<10 ms

All experiments, including both training and deployment, were performed on a single NVIDIA GeForce RTX 3090 GPU. To evaluate the practical applicability of each method, the inference latency was measured as shown in Table 14. The latency is defined as the average time required for the agent to complete one high-level action.

LLMHRL: During the training phase, each high-level action recommendation from the LLM enables the low-level policy to execute multiple steps of actions, thereby avoiding the latency issue caused by querying the LLM for every single low-level action. During deployment, the system relies exclusively on the lightweight student policy, resulting in negligible latency.

LLMTeach: In the training phase, this approach requires an LLM call to provide teacher guidance for every low-level student action. However, like LLMHRL, the deployment phase uses only the trained student network, thus avoiding LLM-related latency.

Inner Monologue & LLM: During deployment, Inner Monologue requires multiple LLM calls to perform iterative self-correction based on environmental feedback, whereas the LLM method only requires a single call. This reliance on online LLM inference introduces significant latency, which severely hinders its practical application in real-time scenarios.

In summary, LLMHRL strikes an effective balance between performance and latency. By confining the LLM inference to the training phase, it ensures that the deployment process is highly responsive with minimal latency.

5. Conclusion

This paper combines option-based hierarchical reinforcement learning with LLM to address the challenges of long-sequence decision tasks in sparse reward environments. LLMHRL guides RL by calculating the difference in the distribution of high-level policy actions between LLM and RL, adding this difference to the loss function. High-level actions are divided into skill and target components to reduce the number of selectable actions. Low-level actions are made up of a skill library to enable different high-level decisions. Experiments show that LLMHRL can learn the LLM’s policy more quickly and gradually learn better policies through environmental interaction. Furthermore, LLMHRL demonstrates deployability to physical robots and achieves robust performance in real-world applications. However, LLMHRL has some limitations, such as fixed options for high-level actions, making it challenging to handle unknown targets. This paper plans to continue improving LLMHRL in future work.

CRedit authorship contribution statement

Qianxi Li: Writing – original draft, Software, Methodology, Data curation, Conceptualization. **Bao Pang:** Writing – review & editing, Supervision, Project administration. **Yong Song:** Supervision, Resources, Funding acquisition. **Hongze Fu:** Writing – original draft, Software, Methodology, Data curation. **Qingyang Xu:** Supervision, Resources, Funding acquisition. **Xianfeng Yuan:** Supervision, Resources, Funding acquisition. **Xiaolong Xu:** Supervision, Resources, Funding acquisition. **Chengjin Zhang:** Supervision, Resources, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is supported by the [National Natural Science Foundation of China](#) (Grant No.62203266).

Data availability

Data will be made available upon request.

References

- [1] B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, et al., Language models are few-shot learners, arXiv preprint, arXiv:2005.14165, 2020, <https://doi.org/10.5555/3495724.3495883>.
- [2] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, et al., Gpt-4 technical report, arXiv preprint arXiv:2303.08774, 2023 <https://doi.org/10.48550/arXiv.2303.08774>.
- [3] R. T. Icarte, T. Q. Klassen, R. Valenzano, S. A. McIlraith, Reward machines: exploiting reward function structure in reinforcement learning, J. Artif. Intell. Res. 73 (2022) 173–208, <https://doi.org/10.1613/jair.1.12440>.
- [4] X. Liu, G. Wang, Z. Liu, Y. Liu, Z. Liu, P. Huang, Hierarchical reinforcement learning integrating with human knowledge for practical robot skill learning in complex multi-stage manipulation, IEEE Trans. Autom. Sci. Eng. 21 (3) (2024) 3852–3862, <https://doi.org/10.1109/TASE.2023.3288037>.
- [5] D. Driess, F. Xia, M. S.M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, et al., Palm-e: an embodied multimodal language model, in: Proceedings of the 40th International Conference on Machine Learning, PMLR, 2023, pp. 8469–8488, <https://doi.org/10.1016/j.cag.2024.01.012>.
- [6] H. Kheddar, Transformers and large language models for efficient intrusion detection systems: a comprehensive survey, Inf. Fusion (2025) 103347.
- [7] L. Chen, Y. Lei, S. Jin, Y. Zhang, L. Zhang, Rlingua: improving reinforcement learning sample efficiency in robotic manipulations with large language models, IEEE Robot. Autom. Lett. 9 (7) (2024) 6075–6082, <https://doi.org/10.1109/LRA.2024.3400189>.
- [8] A. K. Lampinen, I. Dasgupta, S. C.Y. Chan, H. R. Sheahan, A. Creswell, D. Kumaran, J. L. McClelland, F. Hill, Language models, like humans, show content effects on reasoning tasks, PNAS Nexus 3 (7) (2024) pgae233, <https://doi.org/10.1093/pnasnexus/pgae233>.
- [9] R. Wang, D. Zhao, Z. Yuan, I. Obi, B.-C. Min, Prefclm: enhancing preference-based reinforcement learning with crowdsourced large language models, IEEE Robot. Autom. Lett. 10 (3) (2025) 2486–2493, <https://doi.org/10.1109/LRA.2025.3528663>.
- [10] B. Yu, H. Kasaei, M. Cao, L3mvm: leveraging large language models for visual target navigation, in: 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2023, pp. 3554–3560, <https://doi.org/10.1109/IROS55552.2023.10342512>.
- [11] F. Bang, Gptcache: an open-source semantic cache for llm applications enabling faster answers and cost savings, in: Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023), 2023, pp. 212–218, <https://doi.org/10.18653/v1/2023.nlposs-1.24>.
- [12] F. Yang, J. Liu, K. Li, Llm-guided reinforcement learning for interactive environments, Mathematics 13 (12) (2025) 1932, <https://doi.org/10.3390/math13121932>.
- [13] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, Y. Su, Llm-planner: few-shot grounded planning for embodied agents with large language models, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 2998–3009, <https://doi.org/10.1109/ICCV51070.2023.00280>.
- [14] S. Sun, R. Liu, J. Lyu, J.-W. Yang, L. Zhang, X. Li, A large language model-driven reward design framework via dynamic feedback for reinforcement learning, Knowl.-Based Syst. 326 (2025) 114065, <https://doi.org/10.1016/j.knsys.2025.114065>.
- [15] R. Miao, Q. Jia, F. Sun, G. Chen, H. Huang, Hierarchical understanding in robotic manipulation: a knowledge-based framework, in: Actuators, vol. 13, MDPI, 2024, pp. 28, <https://doi.org/10.3390/act13010028>.
- [16] S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, Z. Hu, Reasoning with language model is planning with world model, in: The 2023 Conference on Empirical Methods in Natural Language Processing, 2023, <https://doi.org/10.18653/v1/2023.emnlp-main.507>.
- [17] K. Valmeekam, M. Marquez, A. Olmo, S. Sreedharan, S. Kambhampati, Planbench: an extensible benchmark for evaluating large language models on planning and reasoning about change, Adv. Neural Inf. Process. Syst. 35 (2022) 38975–38987, <https://doi.org/10.5555/3666122.3667815>.
- [18] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, et al., Inner monologue: embodied reasoning through planning with language models, in: 6th Annual Conference on Robot Learning, 2022.
- [19] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al., Chain-of-thought prompting elicits reasoning in large language models, Adv. Neural Inf. Process. Syst. 35 (2022) 24824–24837, <https://doi.org/10.5555/3600270.3602070>.
- [20] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, Y. Iwasawa, Large language models are zero-shot reasoners, Adv. Neural Inf. Process. Syst. 35 (2022) 22199–22213, <https://doi.org/10.5555/3600270.3601883>.
- [21] S. Chakraborty, J. Qiu, H. Yuan, A. Koppel, F. Huang, D. Manocha, A. Bedi, M. Wang, Maxmin-rlhf: towards equitable alignment of large language models with diverse human preferences, in: ICM 2024 Workshop on Models of Human Feedback for AI Alignment, 2024.
- [22] W. Tan, W. Zhang, S. Liu, L. Zheng, X. Wang, B. An, True knowledge comes from practice: aligning llms with embodied environments via reinforcement learning, in: The Twelfth International Conference on Learning Representations, 2024.
- [23] T. Carta, C. Romac, T. Wolf, S. Lamprier, O. Sigaud, P.-Y. Oudeyer, Grounding large language models in interactive environments with online reinforcement learning, in: International Conference on Machine Learning, PMLR, 2023, pp. 3676–3713, <https://doi.org/10.5555/3618408.3618558>.
- [24] A. Szot, M. Schwarzer, H. Agrawal, B. Mazouze, R. Metcalf, W. Talbott, N. Mackraz, R. D. Hjelm, A. T. Toshev, Large language models as generalizable policies for embodied tasks, in: The Twelfth International Conference on Learning Representations, 2023.
- [25] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, A. Anandkumar, Eureka: Human-level reward design via coding large language models, in: The Twelfth International Conference on Learning Representations, 2024.
- [26] T. Xie, S. Zhao, C. H. Wu, Y. Liu, Q. Luo, V. Zhong, Y. Yang, T. Yu, Text2reward: reward shaping with language models for reinforcement learning, in: The Twelfth International Conference on Learning Representations, 2024.

- [27] Z. Zhou, B. Hu, C. Zhao, P. Zhang, B. Liu, Large language model as a policy teacher for training reinforcement learning agents, *Proc. Int. Jt. Conf. Artif. Intell.* (2024) 5671–5679.
- [28] Z. Shen, T. Zhu, Q. Sun, S. Gao, J. Li, Beyond human preferences: exploring reinforcement learning trajectory evaluation and improvement through llms, *Proc. Int. Jt. Conf. Artif. Intell.* (2024).
- [29] A. Brohan, Y. Chebotar, C. Finn, K. Hausman, A. Herzog, D. Ho, J. Ibarz, A. Irpan, E. Jang, R. Julian, et al., Do as I can, not as I say: grounding language in robotic affordances, in: *Conference on Robot Learning*, PMLR, 2023, pp. 287–318.
- [30] R. Hazra, P. Z. Dos Martires, L. De Raedt, Saycanpay: heuristic planning with large language models using learnable domain knowledge, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, 2024, pp. 20123–20133.
- [31] Z. Hu, W. Liu, X. Qu, X. Yue, C. Chen, Z. Wang, Y. Cheng, Divide and conquer: grounding llms as efficient decision-making agents via offline hierarchical reinforcement learning, *Forty-second International Conference on Machine Learning* 2025.
- [32] M. Matthews, M. Samvelyan, J. Parker-Holder, E. Grefenstette, T. Rocktäschel, Hierarchical kickstarting for skill transfer in reinforcement learning, *Proc. Mach. Learn. Res.* (2022) 856–874.
- [33] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, *arXiv preprint arXiv:1707.06347*, 2017.

Author biography



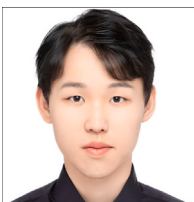
Qianxi Li received the bachelor's degree in Robotics Engineering from School of Mechanical Engineering, University of Shanghai for Science and Technology, Shanghai, China, in 2023. He is currently working toward the master's degree in Control Science and Engineering from School of Mechanical, Electrical and Information Engineering, Shandong University, Weihai, China. His research interests include Deep Reinforcement Learning, Large Language Models and intelligent robot control.



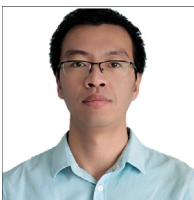
Bao Pang received the M.S. degree in Operations Research and Cybernetics from University of Science and Technology Liao Ning in 2014. He received the Ph.D. degree in Control Theory and Control Engineering from Shandong University in 2020. He is currently a Lecturer at the School of Mechanical, Electrical and Information Engineering, Shandong University, Weihai, China. His current research interests include intelligent robot control, multi-robot collaborative control, Deep Reinforcement Learning and Swarm Intelligence.



Yong Song (Member, IEEE) received the B.S. degree in control science from Shandong University, Weihai, in 2001, and the M.S. and Ph.D. degrees in pattern recognition and intelligent system from Shandong University, in 2008 and 2012, respectively. He is currently a Professor with the School of Mechanical, Electrical and Information Engineering, Shandong University. His current research interests include robot control, machine learning, and swarm intelligence robotics.



Hongze Fu is currently a third-year undergraduate student at School of Mechanical, Electrical and Information Engineering, Shandong University, Weihai, specializing in Robotic Engineering. His research interests include general robotic manipulation, Deep Reinforcement Learning and foundation models in robotics.



Qingyang Xu received the M.S. and Ph.D. degrees in control theory and engineering from Dalian Maritime University, Dalian, China, in 2007 and 2010, respectively. From August 2010 to July 2012, he was a Post-Doctoral Researcher with the Dalian Institute of Chemical Physics, Chinese Academy of Sciences. He is currently an Associate Professor with the School of Mechanical, Electrical and Information Engineering, Shandong University, Weihai, China. His research interests include artificial intelligence, deep learning, and their applications on robot.



Xianfeng Yuan received the Ph.D. degree in control theory and control engineering from Shandong University, China, in 2017. From 2016 to 2017, he was a Visiting Ph.D. student with Oklahoma State University, USA. He is currently an Associate Professor with the School of Mechanical, Electrical and Information Engineering, Shandong University, Weihai, China. His research interests include machine learning, intelligent fault diagnosis, and robotics.



Xiaolong Xu was born in Qingdao, China. He received the B. Eng. degree in mechanical engineering and automation from Qingdao University of Science and Technology, China, in 2011, the M. Eng. degree in mechanical design and theory from Beijing University of Chemical Technology, Beijing, in 2014, and the Ph.D degree in pattern recognition and intelligent systems from Shandong University, Jinan, China, in 2020. He is currently an assistant professor at School of Mechanical, Electrical and Information Engineering, Shandong University, Weihai, China. His current research interests include the robot, embedded systems and wearable devices applied to human-robot collaboration.



Chengjin Zhang received his Ph.D. degree in Industrial Automation from North eastern University in 1997. His research interests include control theory and applications, intelligent robot control, and bioinformatics.